

Ranking versus Regression with Graph Neural Surrogates for Fixed-Budget Design Optimization

Konstantin Cherniak, Rostislav Mardyshev, Egor Shikov,
Peter Shevchenko, Illarion Iov, Gleb Solovov, Dmitry Gilemhanov,
Alena Kropacheva, Anna Kalyuzhnaya, Nikolay Nikitin
NSS Lab, ITMO University, Saint Petersburg, Russia

Working draft — 8 September 2026

Draft for author review. Completed truss experiments; interim network analysis.

[TODO A1: Reconfirm the inherited author list, order, affiliation and contributions for the revised study.]

Abstract

Surrogate-assisted optimization uses learned predictions to decide which candidates deserve an expensive objective evaluation. This decision depends on candidate order, whereas a regression loss penalizes errors in objective values. We investigate whether a pairwise ranking loss improves the final solution obtained under a fixed number of true evaluations. Ranking and regression use the same graph neural network from GAMLET, matched training data and initial weights, and the same evolutionary search procedure. We consider two graph-structured design problems: compliance minimization of planar trusses at constant material volume, and degree-preserving network rewiring for robustness against adaptive node attacks. In the completed truss experiment, comprising five independent offline-data blocks and 120 held-out tasks, RankNet reduces final compliance by 0.42% relative to validation-selected GNN regression and by 12.80% relative to the evolutionary control at 256 true evaluations. Candidate-pool controls support an explanation based on learned screening, while the advantage over regression is substantially smaller than the advantage over unguided search. **[TODO N1: Insert the final five-block network effect and uncertainty after completion and audit; the two-block interim result is reported separately.]** The study evaluates solution quality per online evaluation budget, with offline data costs reported separately, and makes no claim of universal ranking superiority or wall-clock acceleration.

Keywords: surrogate-assisted optimization; learning to rank; graph neural networks; generative design; network robustness; evaluation budget

1 Introduction

Many design problems admit numerous feasible candidates but permit only a limited number of objective evaluations during optimization. A structural candidate may require a numerical solve; a network topology may require a sequence of attacks and connectivity calculations. A surrogate can screen a larger set of proposals and reserve the evaluation budget for a small subset. The relevant prediction is then which candidate to evaluate next, rather than an accurate objective value for every candidate.

This distinction motivates a testable hypothesis: with the graph representation, model capacity, training observations and search budget held constant, learning within-task order can yield better final designs than learning objective values. It does not imply that a regressor cannot rank candidates. Nor does it imply that low pairwise error necessarily identifies the best few candidates in the region visited by an optimizer. A useful test must measure downstream

optimization quality and compare ranking with a competent regression model under the same conditions.

Ranking-based optimization is an established research direction. Recent work addresses ranking in offline model-based optimization and in Bayesian molecular selection, including GNN surrogates [1, 2]. Our contribution is an empirical comparison in two constrained graph design domains, using a common GNN implementation and explicit separation of offline learning, candidate screening and true evaluation. The completed structural experiment includes controls that distinguish the benefit of a learned screening rule from the effect of generating more proposals. A second experiment tests degree-preserving network design, including independent attack scenarios and larger held-out graphs; its final analysis remains pending in this draft.

The scope is optimization across related task contexts. A model is trained on other geometries and loads, or other source networks, and transferred to a held-out task. This is not a claim of transfer between physical trusses and communication networks: each domain has its own fitted models and features. Automated machine learning is another possible application of graph-structured candidate screening, but it is not an empirical benchmark in this revised study.

2 Related work and positioning

Surrogate-assisted evolutionary algorithms combine learned evaluation with search and model management. The survey by Liu et al. [3] separates search strategies from surrogate construction and management for expensive combinatorial optimization. Our experiments examine one component of this design space—the supervised loss used by a fixed screening model—while keeping the remaining search mechanism controlled.

Pairwise neural ranking follows the RankNet formulation of Burges et al. [4]. Ordinal models in evolutionary computation predate the present work [5]; partial-order surrogate methods also address which individuals require exact evaluation [6]. Order-oracle analysis studies optimization using comparisons rather than numerical objective access [7]. An exact order oracle, however, is different from a learned surrogate that can make comparison errors. Theoretical guarantees for the former do not establish the performance of the latter in our discrete design spaces.

The closest recent comparisons make the general ranking hypothesis particularly relevant, but also constrain its novelty. Tan et al. [1] train ranking models for offline model-based optimization using a fixed dataset. Their setting has no adaptive true-evaluation loop during optimization. Tom et al. [2] compare ranking and regression in Bayesian molecule selection, including graph neural networks, and examine the relationship between ordering quality and optimization. Wu et al. [8] use a RankNet-inspired surrogate within a hybrid method for expensive coverage optimization. Our study instead uses a frozen graph surrogate to screen evolutionary proposals on held-out design tasks, while reserving a fixed online budget for true evaluation. We do not present either learning to rank or a GNN ranking surrogate as a new general concept.

Network robustness is also an established optimization application. Smart Rewiring modifies network topology to improve robustness under targeted attacks [9]. Graph-embedding surrogates have already been integrated into evolutionary robustness optimization [10]. ResiNet learns degree-preserving rewiring policies with a graph-based representation [11]. These works motivate the domain; the implementation below is a controlled benchmark, not a reproduction of their algorithms. The classical controls establish internal comparisons, not superiority over specialized state-of-the-art network optimizers.

3 Problem and surrogate-assisted search

3.1 Task-conditioned optimization

Let t denote a task context, $\mathcal{X}_t$ its feasible designs and $f_t : \mathcal{X}_t \rightarrow \mathbb{R}$ an objective to minimize. A design is represented by a graph $G(x)$ with node features, together with a task vector c_t . The offline dataset contains true evaluations on training tasks,

$$\mathcal{D}_{\text{train}} = \{(G(x_{ti}), c_t, f_t(x_{ti})) : t \in \mathcal{T}_{\text{train}}\}. \quad (1)$$

Validation and test tasks are disjoint from training tasks. For a test task, the optimizer may call the objective at most B times, including initialization. The reported solution is

$$\hat{x}_{t,B} \in \underset{x \in \mathcal{A}_{t,B}}{\operatorname{argmin}} f_t(x), \quad (2)$$

where $\mathcal{A}_{t,B}$ contains only truly evaluated candidates. Surrogate scores are never inserted into this archive as if they were objective values. For networks, we train using the cost $f_t(x) = 1 - R_t(x)$ and report the robustness R_t , which is maximized.

3.2 Matched graph models and losses

Both ranking and regression instantiate GAMLET’s actual `SimpleGNNEncoder`, through the common `RepositoryGraphSurrogate` adapter. The encoder uses three GraphSAGE layers [12], 32 hidden channels and mean pooling over active nodes. A task MLP maps six context features through two 16-unit ReLU layers. The pooled graph and context embeddings enter a scalar head with a 32-unit ReLU hidden layer. Dropout and batch normalization are disabled. Inactive truss members are removed before pooling. The models have 8,769 trainable parameters for trusses and 8,417 for networks; within a domain, every loss uses the identical architecture.

For scalar scores $s_i = s_\theta(G(x_i), c_t)$, lower values indicate better designs. In a batch drawn from one training task, let $p_{ij} = 1$ when $f_i < f_j$, $p_{ij} = 0$ when $f_i > f_j$, and $p_{ij} = 1/2$ for a numerical tie. RankNet minimizes

$$\mathcal{L}_{\text{rank}} = -\frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} [p_{ij} \log \sigma(s_j - s_i) + (1 - p_{ij}) \log(1 - \sigma(s_j - s_i))], \quad (3)$$

where $\mathcal{P}$ contains all unordered pairs in the batch. The implementation applies an absolute tie tolerance of 10^{-8} to the training targets. These scores express an ordering, not calibrated objective values or uncertainty estimates.

The regression baselines minimize

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^m (s_i - z_i)^2, \quad z_i = \frac{g(f_i) - \mu_t}{\max(\sigma_t, 10^{-8})}, \quad (4)$$

with $g(f) = f$ or $g(f) = \log f$. The mean and standard deviation come only from the corresponding training task’s offline observations. Network experiments additionally include both transformations with normalization pooled over all training tasks. Test-task objective statistics are not supplied to any model. RankNet compares the within-task standardized targets; their ordering agrees with the untransformed objective apart from numerical tie handling.

Pairwise labels are unchanged by a strictly increasing transformation in exact arithmetic. This removes the requirement to reproduce between-candidate objective magnitudes, but also discards potentially useful information about improvement size. The empirical question is whether this tradeoff helps the screening decision. Matching the architecture and optimizer steps controls model capacity and training allocation; it does not equalize the arithmetic cost of pairwise and pointwise losses.

3.3 Training, validation and candidate selection

All losses share the offline data, initial weights, task order, minibatches and optimizer settings within each model seed. Training uses 80 full epochs, Adam with learning rate 10^{-3} and weight decay 10^{-4} , and gradient-norm clipping at 5. A checkpoint is selected using the same validation criterion for all losses: mean log regret of the single best predicted design in each held-out candidate pool. The regret is $\log(C_{\text{selected}}/C_{\text{pool min}})$ for trusses and $\log(R_{\text{pool max}}/R_{\text{selected}})$ for networks. Pool optima here are optima of a finite validation pool, not of the design problem. The main regression recipe is selected by mean validation regret across model seeds, separately in each offline-data block, before test search.

Each search begins with 16 shared truly evaluated candidates. The parent archive retains the 16 best true evaluations. At each iteration, mutation generates a pool of 48 feasible, unevaluated candidates. The surrogate selects the three lowest scores, and one further candidate is sampled uniformly from the remaining pool. Only those four candidates are evaluated; the archive is then updated with their true values. Main-search models remain frozen. Initialization, archive size, mutation, exploration and budget accounting are identical between ranking and regression.

The evolutionary control (EA) evaluates four generated offspring directly. Random search or random restarts provide a second model-free control. Additional experiments select four candidates randomly from a pool of 48, separating proposal-pool size from learned screening. The network benchmark also includes single-swap hill climbing around the best evaluated graph. These are explicitly defined controls; a lower value of B does not make their wall-clock costs equal.

4 Benchmarks and experimental protocol

4.1 Generative design of planar trusses

The structural task minimizes compliance of a pin-jointed, small-displacement planar truss at constant material volume. Twelve joints lie on a 4×3 grid. The ground structure has 29 candidate bars: 23 mandatory horizontal, vertical and diagonal members, and six optional opposite diagonals. All translations on the left boundary are fixed. Crossing bars share a joint only at a defined endpoint. Mandatory triangular cells restrict the search to a stable family; the benchmark primarily tests discrete member sizing with limited topology variation.

Mandatory-member genes take values in $\{1, 2, 3, 4\}$ and optional genes in $\{0, 1, 2, 3, 4\}$. For member length ℓ_e , cross-sectional areas satisfy

$$A_e = \frac{V g_e}{\sum_j \ell_j g_j}, \quad \sum_e \ell_e A_e = V, \quad (5)$$

with $V = 0.03 \text{ m}^3$ and Young’s modulus $E = 210 \text{ GPa}$. Proportional genotypes are deduplicated by their greatest common divisor. Using the reduced compatibility matrix D , the evaluator computes

$$K = D^\top \text{diag}(EA_e/\ell_e)D, \quad Ku = F, \quad C = F^\top u. \quad (6)$$

Compliance is measured in N m and must be positive and finite, with relative solve residual at most 10^{-7} .

Task contexts independently vary span over $[3, 6] \text{ m}$, height over $[1, 2.5] \text{ m}$, load magnitude over $[50, 150] \text{ kN}$, load angle over $[-0.35, 0.35]$ radians from downward, and the upper-node load fraction over $[0.15, 0.85]$, using uniform distributions. The load is split between the upper and lower right joints. Mutation changes one to three genes, with uniform crossover between elite parents applied with probability 0.25.

The GNN receives the line graph of active members: two nodes are adjacent when the physical members share an endpoint. Its 17 node features describe normalized endpoint coordinates,

direction, length, area, volume fraction, endpoint loads and support indicators. The context vector contains scaled span, height and force, sine and cosine of the load angle, and the load fraction. Displacements, stresses and compliance are excluded from input features.

Five independent offline-data blocks each contain 24 training, eight validation and 24 test contexts. Training and validation use 512 designs per context, shared by all losses. Canonical offline genotypes are disjoint across the generated contexts within a block. There are five model seeds and ten search seeds per test task, with batch size 64 during training. The primary budget is 256 true evaluations; endpoints 32 and 96 are available from the same trajectories. The completed main series contains 20,400 searches on 120 held-out tasks. It consumes 81,920 offline training/validation evaluations and 5,222,400 online evaluations; diagnostic and audit evaluations are separate.

4.2 Degree-preserving network design

Here a candidate is a connected, simple, undirected, unweighted graph. Feasible double-edge swaps preserve each vertex degree and the total edge count. Relative to its source graph, a design may contain at most $\lceil 0.2|E| \rceil$ new edges. This bound counts new edges, not the number of swap operations. Offspring use one to three feasible swaps.

For each attack scenario, repeatedly delete a vertex of maximum *current* degree, updating degrees after every deletion. Ties follow a fixed random priority permutation. Let $L_{s,q}(G)$ be the largest connected component after q deletions in scenario s . The objective is

$$R(G) = \frac{1}{Sn^2} \sum_{s=1}^S \sum_{q=1}^n L_{s,q}(G), \quad S = 4. \quad (7)$$

The same four scenarios are used for all candidates and methods within a task. Thus one true evaluation is an exact calculation on a finite scenario bank, not the expectation over every possible tie ordering. A heap computes the adaptive attack order; reverse insertion with union-find computes component sizes. Independent NetworkX calculations audit the resulting attack curves.

Source graphs are Barabási–Albert networks with attachment parameter 3 and connected Erdős–Rényi $G(n, m)$ graphs with matched $m = 3(n - 3)$. Training and in-distribution testing use $n \in \{50, 100\}$. Vertex labels are randomly permuted. All variants of a source graph remain in its split, and the source manifest is checked for repeated isomorphic graphs across blocks and against the excluded pilot.

Six node features contain scaled degree, degree relative to its mean, mean and standard deviation of neighbor degrees relative to mean degree, clustering coefficient, and a constant. Six context features encode graph size, mean degree, maximum degree, relative degree dispersion, permitted edit fraction, and a constant. Features exclude attack priorities, node identifiers and attack outcomes. Although degrees are fixed during rewiring, adjacency and neighborhood features can change.

The fixed campaign comprises five independent offline-data blocks. Each contains 32 training, 12 validation and 32 in-distribution test source graphs, balanced across family and size, plus eight test graphs with 200 vertices for size extrapolation. Each training/validation source graph supplies 128 designs. Two model seeds, three search seeds and minibatches of 32 are used. Five losses times two model seeds, plus four model-free controls, give 14 variants per task/search seed. The primary budget is 96 evaluations and the secondary endpoint is 192 on the same trajectory. The planned complete series contains 8,400 searches, 28,160 offline evaluations and 1,612,800 online evaluations.

At both budget endpoints, the already selected graph is evaluated on 32 fresh priority scenarios, shared across methods within a task. These scores cannot alter search, checkpoint selection or the selected graph. Across the complete planned series this check requires 537,600 additional

Table 1: Completed truss experiment at 256 true evaluations: five offline-data blocks and 120 held-out tasks. Negative compliance changes favor RankNet. Intervals are exploratory hierarchical bootstrap intervals, not task-conditional network intervals.

Comparator	Δ_C (%)	Exploratory 95% interval (%)
Validation-selected GNN regression	-0.416	$[-0.592, -0.254]$
GNN MSE	-0.620	$[-0.854, -0.387]$
GNN log-MSE	-0.366	$[-0.476, -0.264]$
EA	-12.795	$[-13.256, -12.332]$
Random search	-24.884	$[-25.606, -24.138]$

attack trajectories; reference audits incur further calls. **[TODO N1: Confirm completion, all five block audits and the final realized counts.]** The protocol and allocation were fixed after a pilot, which is excluded; the campaign was not externally preregistered.

4.3 Effect estimates and statistical units

For a task and comparator, we first average paired log objective ratios across matched model and search seeds. We then average over tasks and equally over offline-data blocks; network tasks are weighted equally within family/size strata, with equal stratum weights. The reported effects are

$$\Delta_C = 100 \left[\exp(\overline{\log(C_{\text{rank}}/C_{\text{base}})}) - 1 \right], \quad (8)$$

$$\Delta_R = 100 \left[\exp(\overline{\log(R_{\text{rank}}/R_{\text{base}})}) - 1 \right]. \quad (9)$$

Negative Δ_C and positive Δ_R favor ranking. Repeated seeds are not independent tasks; a model-free run shared by two model seeds is not counted as two independent observations. Ratios compare attained objectives, not regret relative to an unknown global optimum.

Truss intervals use 4,000 exploratory hierarchical bootstrap samples over offline blocks and tasks. The network protocol specifies 10,000 hierarchical bootstrap samples over blocks and source graphs within strata. With only five top-level blocks, these intervals are approximate and must be read alongside individual block effects.

The network statistical addendum was written after inspecting initial effects, so its tests are exploratory. Exact two-sided sign tests count task wins after seed aggregation, conditional on the fitted models. They test the probability of a positive task effect, not directly the magnitude of the mean improvement. Holm adjustment covers 36 comparisons: nine comparators, two budgets and two scenario banks. This correction does not account for repeated interim inspection. Tests on independent offline-block effects address a different level of generalization. Wilcoxon signed-rank and parametric block tests are retained as sensitivity analyses with their respective assumptions, rather than used to select the smallest p -value. No task is removed as an outlier.

5 Results

5.1 Completed structural experiment

Table 1 reports the completed five-block results. RankNet attains 0.42% lower compliance than the validation-selected regression recipe. The reduction is 0.62% relative to MSE and 0.37% relative to log-MSE. All five block effects for the primary comparison favor RankNet, ranging from approximately -0.72% to -0.24%. The exploratory interval is compatible with a small, consistent advantage in this benchmark, but its width does not establish generality across arbitrary training datasets.

Table 2: Completed truss common-pool diagnostics, averaged in the saved five-block report. Precision at three is the fraction of the three best true candidates recovered among the three lowest predicted scores. The final column compares the mean compliance of predicted selections with random selection; negative values are better. These are descriptive diagnostics on separate tasks, not additional main-test replicates.

Pool location	Loss	Pair accuracy	Precision at 3	Compliance change (%)
Initial	MSE	0.904	0.642	−26.43
Initial	log-MSE	0.902	0.639	−26.43
Initial	RankNet	0.900	0.625	−26.48
Near EA solution	MSE	0.859	0.386	−7.14
Near EA solution	log-MSE	0.864	0.389	−7.18
Near EA solution	RankNet	0.867	0.417	−7.33

The comparison with model-free search has a different scale: compliance is 12.80% lower than EA and 24.88% lower than random search. Most of the observed improvement over EA is therefore associated with using a useful learned screening model; switching its loss from regression to ranking gives a smaller additional gain. A comparison only against EA would not substantiate the ranking-versus-regression hypothesis.

5.2 Why screening helps, and why ranking helps less

Separate diagnostic tasks reuse the trained models from each block, with eight additional contexts, three model seeds and five search seeds per block. Pool sizes are 4, 12, 48 and 192. Controls include random selection from the same pool, untrained models and models trained on shuffled labels. At pool size 48, RankNet has 13.61% lower compliance than random pool selection, 21.07% lower compliance than the untrained model and 15.32% lower compliance than the shuffled-label control. When the pool contains only four candidates, all four are evaluated and RankNet and EA produce identical outcomes in the paired control. These observations support learned candidate selection as an explanation for the improvement at a fixed true-evaluation budget.

Common-pool diagnostics further distinguish global pair ordering from selecting a few good local proposals (Table 2). All oracle calls used to score these pools occur after the search and provide no feedback to it. On initial pools, RankNet has slightly lower pair accuracy and precision at three than either regressor. Near EA solutions, its precision at three is higher. This pattern is consistent with a modest advantage in local screening, but does not establish that improved global ranking accuracy mediates every downstream gain. Differences in visited candidates and accumulated archive quality can also affect the outcome.

5.3 Network robustness: explicitly interim evidence

This subsection uses only the frozen, audited two-block interim snapshot. It contains 64 in-distribution source networks, with six matched model/search-seed combinations averaged within each source network. It must not be presented as the outcome of the complete five-block campaign.

At 96 true evaluations, RankNet improves robustness relative to validation-selected regression by 1.167%. The conditional stratified bootstrap interval is [0.948%, 1.390%] with the trained models held fixed. Mean task robustness is 0.295139 for RankNet and 0.292076 for regression, an absolute difference of 0.0030625. RankNet wins on 55 of 64 task-level comparisons; the exact two-sided sign-test p -value is 3.54×10^{-9} and the Holm-adjusted value is 2.83×10^{-8} . These values concern task wins conditional on these models, rather than reproducibility after retraining on new offline data.

Table 3: Exploratory two-block network snapshot: 64 in-distribution source graphs. RankNet versus validation-selected GNN regression. Positive robustness changes favor RankNet. Holm correction is across 36 task-conditional sign tests; the unadjusted offline-block sign-test p is 0.5 in every row.

B	Scenario bank	Δ_R (%)	Task wins	Holm p
96	Four search scenarios	+1.167	55/64	2.834×10^{-8}
96	32 fresh scenarios	+1.027	50/64	1.414×10^{-5}
192	Four search scenarios	+1.151	54/64	9.982×10^{-8}
192	32 fresh scenarios	+0.982	51/64	7.524×10^{-6}

Both offline-block effects are positive, but the exact two-sided block sign test gives $p = 0.5$, as does the block sign-flip test. Even with five nonzero block effects, the smallest attainable two-sided exact sign-test value is $2/2^5 = 0.0625$. A small task-level p -value cannot remove this limitation. The test direction and number of blocks are not changed in response to interim significance.

Fresh-scenario gains remain positive in the interim snapshot (Table 3), suggesting that the observed benefit is not entirely specific to the four search scenarios. At budget 96 on the search bank, the gains over EA, random pool selection and hill climbing are 4.932%, 4.767% and 1.731%, respectively. These remain exploratory control comparisons. An additional audit independently checks 1,536 endpoint graphs in the primary ranking/regression comparison; all four search-bank attack curves agree with NetworkX, with zero discrepancy, and the complete fresh bank is recomputed.

5.4 Complete network campaign: pending

[**TODO N1:** Replace this paragraph with the audited five-block aggregate: 160 in-distribution tasks, per-block effects, realized budgets, all regression recipes and classical controls. Do not extend the two-block estimate by relabeling its sample size.]

[**TODO N2:** Insert the final task-conditional sign tests, Holm corrections, effect intervals, task win counts and separate offline-block sensitivity tests using the frozen statistical addendum. Preserve its exploratory status.]

[**TODO N3:** Report the 40 held-out 200-node graphs separately, including family-specific and fresh-scenario effects. Do not pool these extrapolation tasks into the primary 50/100-node result.]

[**TODO N4:** Add convergence versus true evaluations and final five-block network tables/figures from the audited aggregate. Show primary 96 and secondary 192 endpoints without treating trajectory prefixes as independent runs.]

6 Discussion and limitations

The completed structural results support a narrow practical conclusion: under the tested representation, training distribution and search procedure, pairwise training provides a small improvement over matched GNN regression. The larger difference from EA is supported by proposal-pool and label controls, which indicate that useful screening directs true evaluations toward better candidates. The ranking-specific improvement should remain visible as a separate effect, rather than being inferred from the much larger surrogate-versus-EA difference.

The distinction between local screening and global prediction matters. An optimizer repeatedly changes the proposal distribution by updating its elite archive. A model can order a broad offline pool well yet miss the best few candidates near a good incumbent. Conversely, a model with slightly lower global pair accuracy can still select useful local improvements. Our

common-pool results are compatible with this explanation, but a ranking loss alone does not guarantee top-of-pool accuracy. Pairwise training also gives nearly tied and widely separated pairs similar target labels, which may be undesirable in other objective landscapes.

Offline training is a substantive resource. The experiment assumes its data can be reused across related new tasks. Equal online evaluation budgets isolate selection efficiency after that investment; they do not establish a reduction in total data requirements for a single task, or faster execution on these small evaluators. The network objective itself uses four attack trajectories per call. Offline labels, fresh-scenario checks and reference audits must be accounted for separately rather than silently included in the search budget or omitted from reproducibility records.

The trusses use linear axial mechanics, fixed material volume and a restricted ground structure. Stress limits, buckling, fatigue, geometric nonlinearity and manufacturing feasibility are outside the model. Thus these results concern a numerical generative-design benchmark, not a certified structural design method. Network instances are synthetic, unweighted and limited to two source families; degree-preserving edits and one adaptive attack policy further bound their scope. Extrapolation to 200 vertices requires the pending analysis and cannot establish transfer to arbitrary large real networks.

Five offline-data blocks provide replication, but limited information about retraining variability. Repeated model and search seeds improve characterization within a task, not the number of independent task contexts. Statistical analyses are exploratory, and hierarchical intervals with few blocks can be overly reassuring. The network interim evidence is conditional on two fitted blocks and must remain distinct from the complete result. No global optimum is known, no complete comparison with specialized network optimizers is made, and no claim of universal ranking superiority follows.

7 Conclusion

We evaluate ranking and regression as alternative training objectives for the same graph surrogate in fixed-budget design optimization. The completed truss experiment yields a 0.42% compliance reduction relative to validation-selected GNN regression at 256 true evaluations, compared with a 12.80% reduction relative to EA. Diagnostic controls support learned screening as the main source of the larger surrogate benefit and show that global pair accuracy alone does not explain optimization performance. **[TODO N1: Complete the cross-domain conclusion only after the five-block network campaign and its audits finish.]** The appropriate claim is an empirical, domain-bounded test of order-based screening, with offline learning costs and statistical replication levels made explicit.

Reproducibility and declarations — draft

The implementation reuses the GAMLET repository at <https://github.com/ITMO-NSS-team/GAMLET>. Experiment additions and their frozen snapshots are currently present in the local workspace. The paper repository contains the numerical evidence snapshots used for this draft, source hashes and build instructions. **[TODO R1: Publish or archive an immutable version of the experimental code, configurations, generated data and analysis outputs, and replace this provisional statement with verified availability links. The legacy repository HEAD alone does not identify the new experiments.]**

[TODO A2: Authors to verify funding applicable to the new experiments, sponsor role, conflicts of interest, contributions, correspondence details and any venue-required declarations. Do not assume that declarations in the archived manuscript still apply.]

Research-assistance record for author review. OpenAI Codex assisted with experimental code and analysis in the associated workspace, and with drafting and local consistency checks for this revision. The Scientific Agent Skills workflow informed statistical and writing

checks [13]. **[TODO A3: Authors to verify all affected content and adapt the disclosure to the chosen venue; no completed human verification or final author approval is asserted here.]**

References

- [1] Rong-Xi Tan, Ke Xue, Shen-Huan Lyu, Haopu Shang, Yao Wang, Yaoyuan Wang, Fu Sheng, and Chao Qian. Offline model-based optimization by learning to rank. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/768c19273e20fa09147885d03da7550f-Abstract-Conference.html.
- [2] Gary Tom, Stanley Lo, Samantha Corapi, Alán Aspuru-Guzik, and Benjamin Sanchez-Lengeling. Ranking over regression for Bayesian optimization and molecule selection. *APL Machine Learning*, 3:036113, 2025. doi: 10.1063/5.0272663. URL <https://arxiv.org/abs/2410.09290>.
- [3] Shulei Liu, Handing Wang, Wei Peng, and Wen Yao. Surrogate-assisted evolutionary algorithms for expensive combinatorial optimization: a survey. *Complex & Intelligent Systems*, 10:5933–5949, 2024. doi: 10.1007/s40747-024-01465-5.
- [4] Chris J. C. Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. Technical Report MSR-TR-2005-06, Microsoft Research, 2005. URL <https://www.microsoft.com/en-us/research/publication/learning-to-rank-using-gradient-descent/>.
- [5] Tómas Philip Rúnarsson. Ordinal regression in evolutionary computation. In *Parallel Problem Solving from Nature—PPSN IX*, pages 1048–1057, 2006. URL <https://iris.hi.is/en/publications/ordinal-regression-in-evolutionary-computation-parallel-problem-s/>.
- [6] Vanessa Volz, Günter Rudolph, and Boris Naujoks. Surrogate-assisted partial order-based evolutionary optimisation. arXiv:1611.00260, 2016. URL <https://arxiv.org/abs/1611.00260>.
- [7] Aleksandr Lobanov, Alexander Gasnikov, and Andrei Krasnov. Acceleration exists! optimization problems when oracle can only compare objective function values. arXiv:2402.09014, version 3, 2024. URL <https://arxiv.org/abs/2402.09014v3>.
- [8] Tongyu Wu, Changhao Miao, Yuntian Zhang, Fang Deng, and Chen Chen. A RankNet-inspired surrogate-assisted hybrid metaheuristic for expensive coverage optimization. arXiv:2501.07375, version 2, 2025. URL <https://arxiv.org/abs/2501.07375v2>.
- [9] V. H. P. Louzada, F. Daolio, H. J. Herrmann, and M. Tomassini. Smart rewiring for network robustness. *Journal of Complex Networks*, 2013. doi: 10.1093/comnet/cnt010. URL <https://arxiv.org/abs/1303.5269>.
- [10] Shuai Wang, Jing Liu, and Yaochu Jin. Surrogate-assisted robust optimization of large-scale networks based on graph embedding. *IEEE Transactions on Evolutionary Computation*, 2019. doi: 10.1109/TEVC.2019.2950935. Early online publication, 7 November 2019.
- [11] Shanchao Yang, Kaili Ma, Baoxiang Wang, Tianshu Yu, and Hongyuan Zha. Learning to boost resilience of complex networks via neural edge rewiring. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=moZv0x5cxe>.

- [12] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017. URL <https://arxiv.org/abs/1706.02216>.
- [13] Timothy Kassis, Vinayak Agarwal, Yuhuan He, Darshil Patel, and Aubrey M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents. arXiv:2609.00065, 2026. URL <https://arxiv.org/abs/2609.00065>.